Table of Contents

Table of Contents **0**
1 IAP Overview **1**
1.1 IAP Introduction 1
1.2 IAP Driver 1
2 Transplant Procedure **3**
2.1 IAP Transplant 3
2.1.1 Migration File 4
2.1.2 IAR Software Configuration 5
2.1.3 KEIL Software Configuration 7
2.2 APP Transplant 8
2.2.1 Migration File 8
2.2.2 IAR Software Configuration 9
2.2.3 KEIL Software Configuration 13
3 Interrupt Vector Table **13**
4 Run IAP Program **15**
4.1 Hyper Terminal Configuration 15
4.2 Run IAP Program 16
5 IAP Program Menu **16**
5.1 Download image files to internal flash memory 17
5.2 Read data from internal flash to .bin file 17
5.3 Execute a new program 17
5.4 Unprotect write protection 18
6 Target chip IAP summary of implementation **18**
7 User program rules **19**
8 How to use IAP driver **19**
9 YMODEM Protocol **20**
10 Related ASCII table **22**

1 IAP Overview

IAP (In Application Programming) is a kind of technology, the user's program updates some parts of user flash in the application, the purpose is to make it more convenient to use the reserved communication interface (UART) to update the firmware of the product program. When designing firmware, two project codes are written. The first project code does not execute normal function codes, only receives data through some communication interface, and updates the second project code. The second project code is the real function code. Both parts of the project code exist in user flash at the same time. When the chip is reset, the first part of the project code runs. It does the following:

1, Do you need to update the second project code?

2, If there is no need to update, jump to 4.

3, Perform update operations.

4, Jump to second code, execute.

The first project code must be programmed through SWD. The second project code can be programmed using the IAP function of the first project code, or it can be programmed together with the first project code using the SWD interface. Later, when the program needs to be updated, it can be updated using the IAP function of the first project code.

For the target chip, because its interrupt vector table is located in the lowest location area of program memory, in order to make the first project code correctly respond to interrupt events, it is usually arranged that the first project code is located in the start area of the flash and the second project code is located after the first project code.

Since the target chip does not have the hardware interrupt vector table remapping function, it is necessary to remap the interrupt vector table of the second project code by software programming.

1.1 IAP Introduction

The IAP driver must be programmed to the beginning of the flash memory via the SWD interface using any development tool selected by the user. The driver then uses the UART to download the binary file from the HyperTerminal on the PC to the internal flash memory of the chip, and then go to execute the download program.

1.2 IAP Driver

◆ IAP driver contains the following source files：

- *Main.c*: this file contains USART initialization and RCC（reset clock controller）

settings, then executes the main menu from the *common.c* file.

- Common.c: including display and main menu. The main menu has a series of operating options: loading new binaries、executing loaded binaries、Write-protect the memory pages that the user wants to load their own binary files (if those pages are write-protected).
- *Ymodem.c* and *download.c*： They are used to receive data from the HyperTerminal (using the YMODEM(a) protocol) and then load the data into RAM inside the target chip. When an error occurs in receiving data, the error message "Failed to receive the file" is displayed, and if the data is successfully received, it is programmed to the appropriate address of the internal flash Memory. Data integrity is detected by comparing internal RAM data with internal flash data. If there is any data difference, the error message "Verification failed" is displayed. Other error messages are also displayed, such as: the image file is larger than the allowed memory space, the user aborted the task, and so on.

  The specific tips are as follows:

  1. The program was successfully programmed: Programming Completed Successfully!
  2. Insufficient memory space: The image size is larger than the allowed memory size!
  3. The data in RAM is different from the data in flash Memory: Verification failed!
  4. User abort program: Aborted by user.
  5. Receive failure: Failed to receive the file!
- *Ymodem.c* and *Upload.c*: They are used to extract the data in the APP code area of flash (using the YMODEM protocol) and save the data to the *UploadedFlashImage.bin* file.
- Target chip firmware library.

◆ After the chip is powered on, the user selects whether to send '5' through HyperTerminal to select whether to run the user program or perform IAP to reprogram.

- Switch to the user program without sending '5' on reset.
- Send '5' on reset to display the IAP main menu.

◆ During the execution of the user program, if it is detected that the host computer sends '6', it jumps to execute the IAP program and displays the main menu in the host computer.

See Table 1 on details for using the host computer to send data and enter the IAP mode.

Note:

(a) The YMODEM protocol transmits 1024 bytes as a data block: it is transferred to the data block of the target internal RAM, and an error check is also performed: comparing whether the RAM storage data is consistent with

the flash received data. If a NAK response occurs, the data block has not been successfully received. See Section 7 for more details on TMODEM.

- The IAP flow chart is as follows:

  Note: The prompt information only lists a part, please refer to the code.

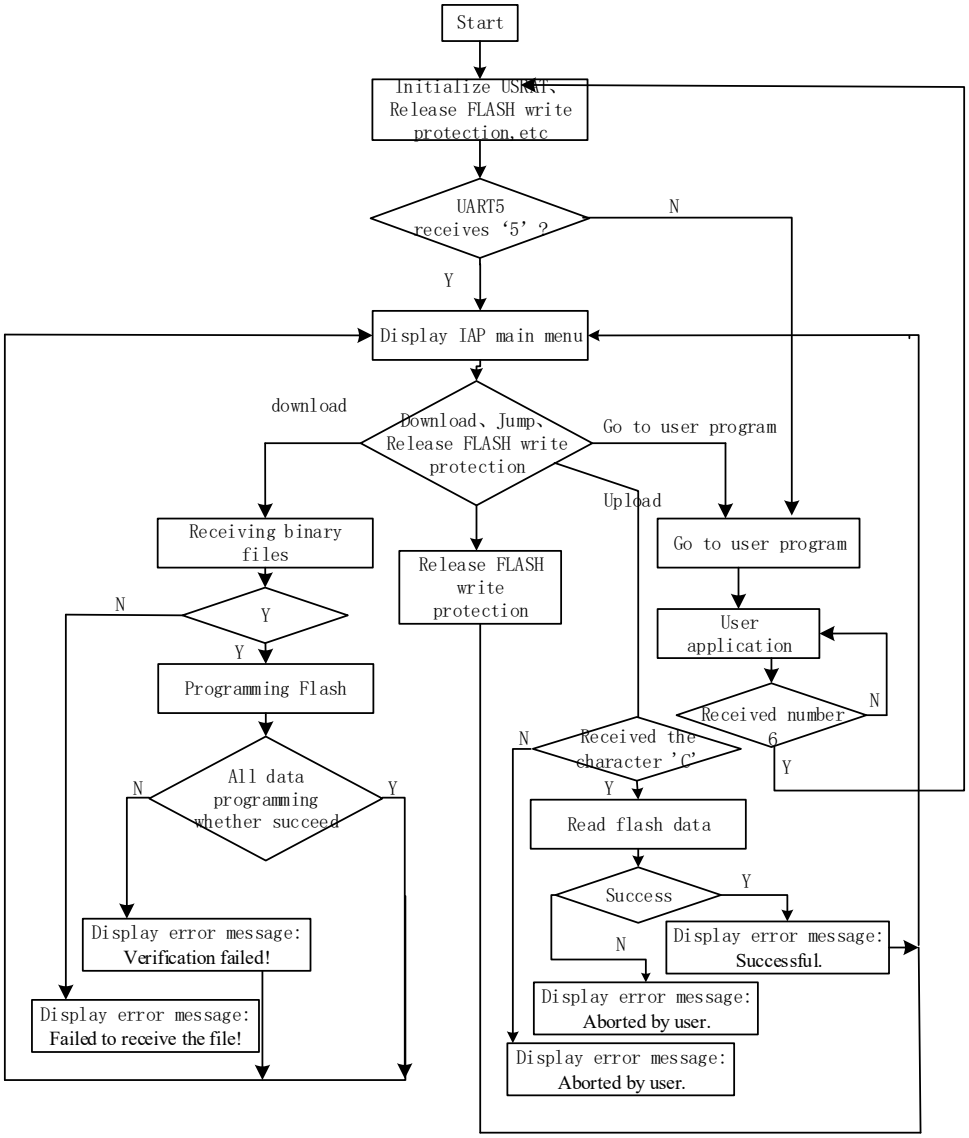


Figure 1. IAP driver flow chart

2 Transplant Procedure

2.1 IAP Transplant

2.1.1 Migration File

(1) Transplant source files

Source files	Functional description
Main.c	Clock initialization, UART initialization, Release flash write protection, IAP and APP jumps.
Ymodem.c and *download.c*	a) They are used to receive data from the HyperTerminal (using the YMODEM(a) protocol) and then load the data into the RAM inside the target chip. When an error occurs in receiving data, the error message "Failed to receive the file" is displayed, and if the data is successfully received, it is programmed to the appropriate address of the internal flash Memory. Data integrity is detected by comparing internal RAM data with internal flash data. If there is any data difference, the error message "Verification failed" is displayed. Other error messages are also displayed, such as: the image file is larger than the allowed memory space, the user aborted the task, and so on. b) The program will calculate the required flash size according to the bin file, and then erases some pages of flash. c) The program will save the data to be programmed to RAM and then program it to the flash area. Compare flash and RAM data.
Ymodem.c and *Upload.c*	a) They are used to extract the data in the APP code area of flash (using the YMODEM protocol) and save the data to the UploadedFlashImage.bin file. b) The name of Bin file can be modified in the Upload.c file. c) Flash start address of reading area: ApplicationAddress (start address of user code). d) Flash size of reading area: FLASH_IMAGE_SIZE (0x4000-0x7FFFF)
Common.c	Include display features and main menu. The main menu gives a list of operating options: loading new binaries, executing loaded binaries, and unprotecting the memory pages that the user wants to load their own binaries (if those pages are write protected).
target_isr.c	Maintenance interrupt vector table.

startup_target.s target_flash.icf	1. Execute the reset handler and Chip Init function which are in the startup file. 2. Locate the Chip Init function in the 0xc0 position.

(2) Modify the starting address of the user application code.

The starting address of the APP is defined in the file *common.h* as follows:

```
#define ApplicationAddress 0x00004000.
```

Modify this value to locate the user application to the desired address.

(3) Modify the area of the internal flash memory for reading data from.

The read area is determined by the macro definition FLASH_IMAGE_SIZE in common.h:

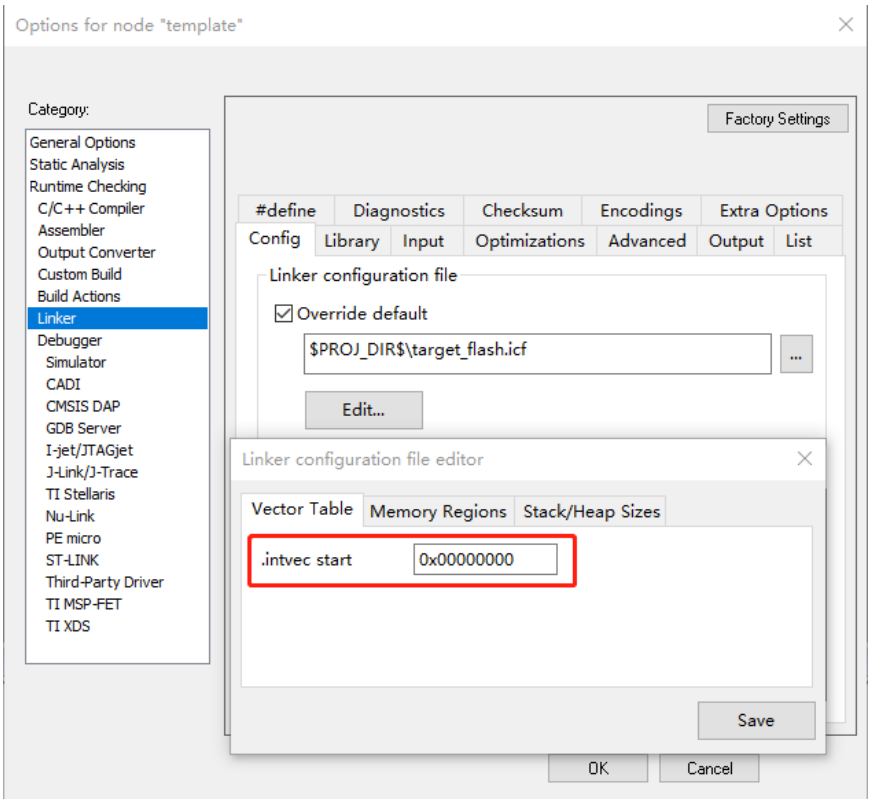
```
#define FLASH_IMAGE_SIZE   (uint32_t) (FLASH_SIZE - (ApplicationAddress - 0x0000000))
```

Among them, FLASH_SIZE is the size of flash, and ApplicationAddress is the starting address of the user program.

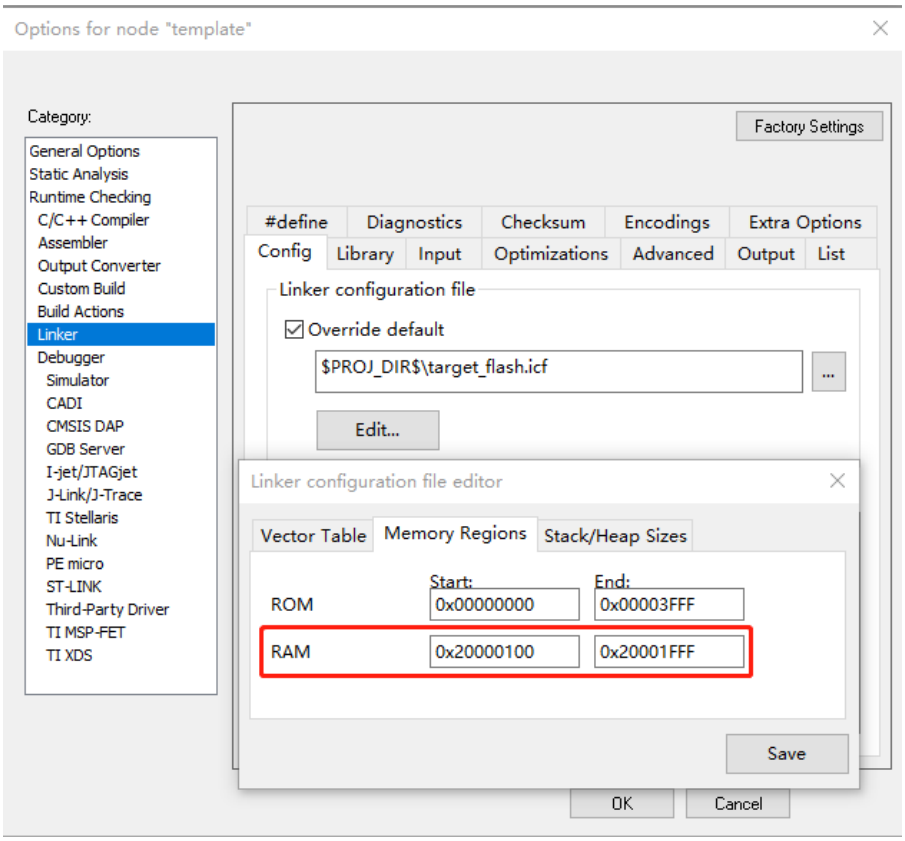
(4) Selecting UART5 as the peripheral, configure UART5 baud rate to 115200, and configure other parameters (data length is 8bit, transmission mode is LSB), input and output enable.

2.1.2 IAR Software Configuration

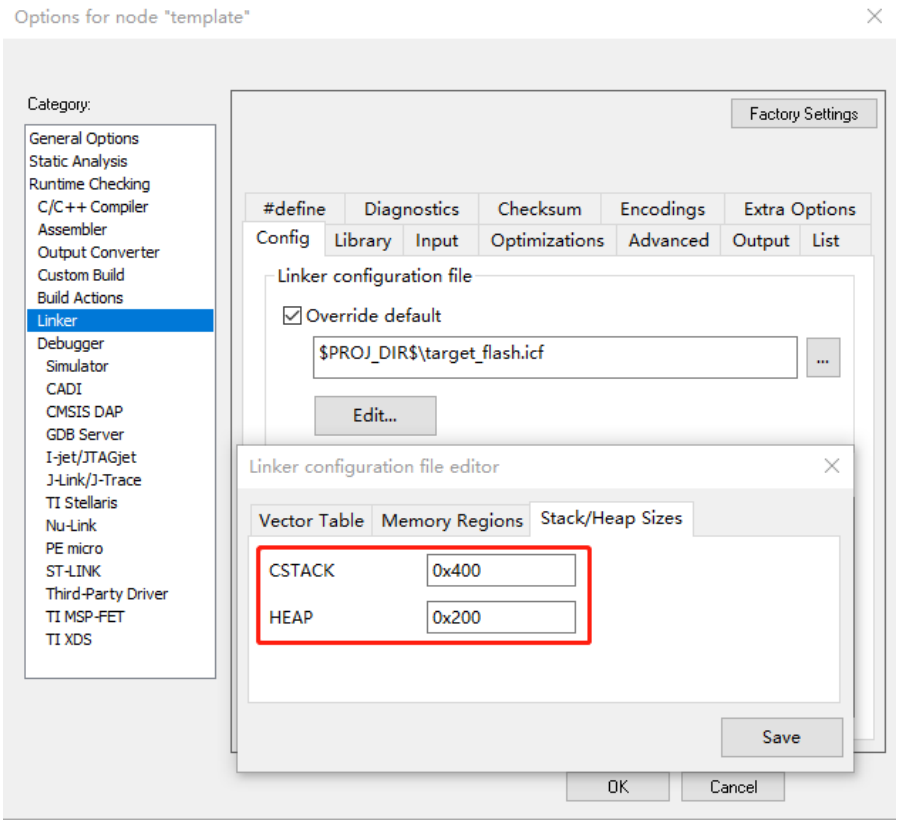
（1） Set the IAP interrupt vector offset address.



（2） Set the size of the ROM and RAM.



（1） Set the stack size.

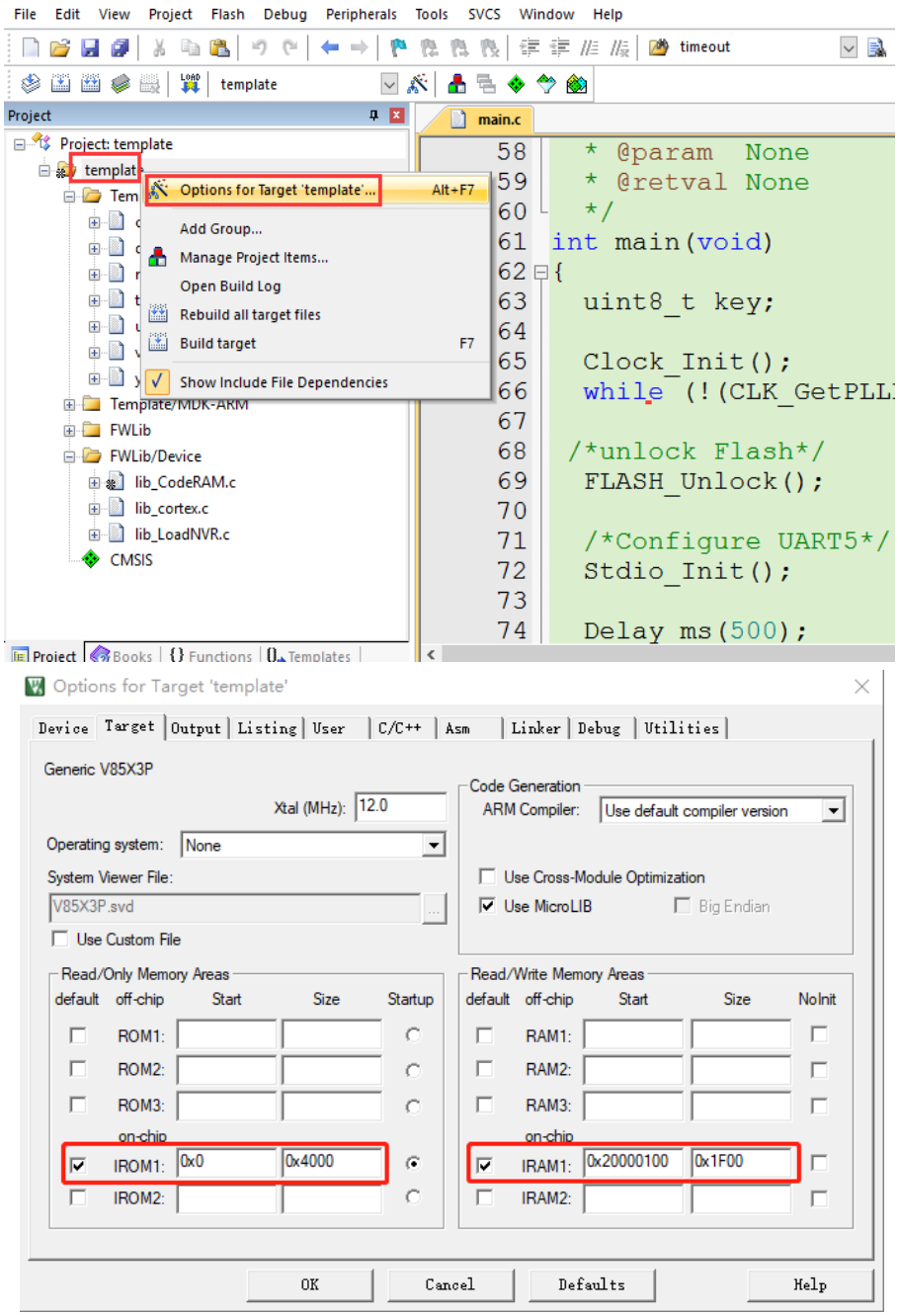


2.1.3 KEIL Software Configuration

（1） Set the stack size (Note: The stack size is set in the startup_target.s file).

```
Stack_Size      EQU     0x000000400
Heap_Size       EQU     0x00000200
```

（2） Set the size of ROM,RAM(Click on the project, right click->option)



2.2 APP Transplant

2.2.1 Migration File

1. Transplant source files

Source files	Functional description

Main.c	Clock initialization, UART initialization, jump between IAP and APP.
IAR: startup_target.s target_flash.icf KEIL: startup_target.s	Startup file. No Chip Init function in the startup file.

2. Please refer to the Bootloader_APP sample code.

（1） The starting address of the IAP is defined in the Main.c function;

```
#define FLASH_IAP_ADDR 0x00000000
```

（2） define a jump function;

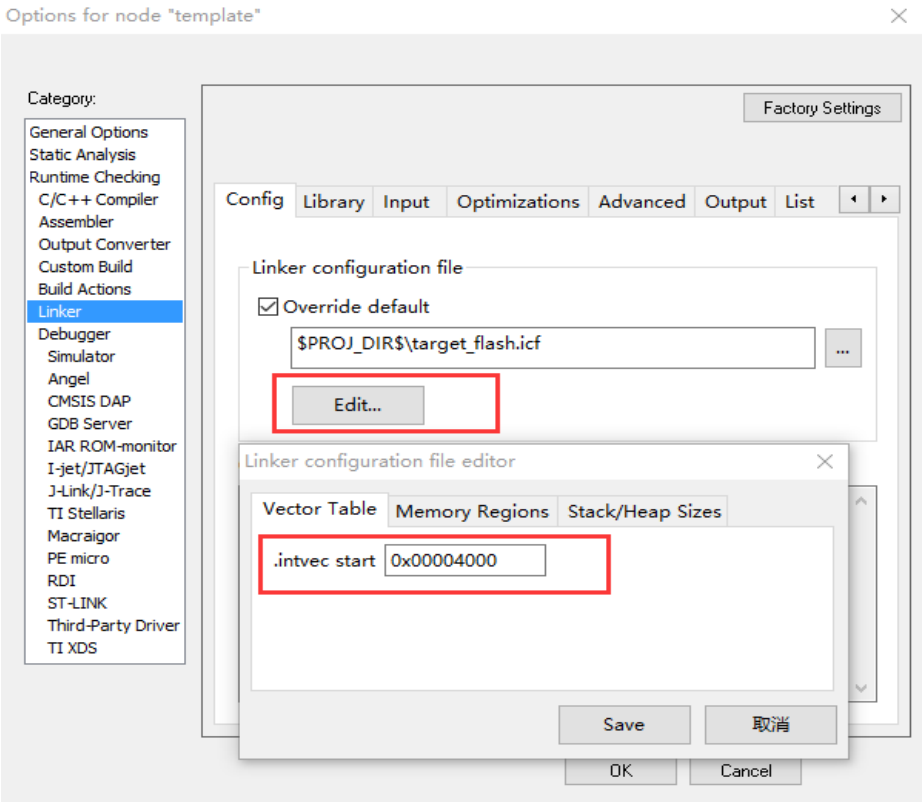
```
typedef void (*IapFun) (void);
IapFun JumpToIap;
```

（3） APP-->IAP program jump implementation. The sample code implementation is to wait for the serial port to receive data in while (1). If the serial port receives '5', it jumps to execute the IAP program.

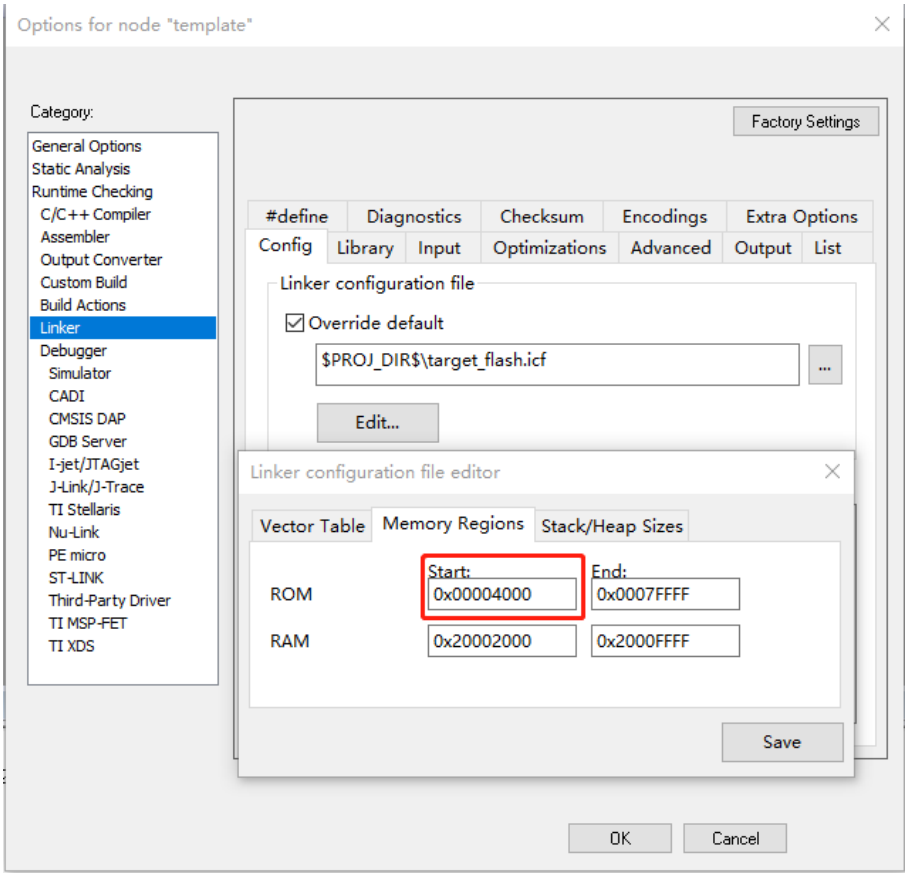
```
void main(void)
{
  // Initial configuration
  while (1)
  {
    /*If UART receive '6', jump to IAP program*/
    if (KeyJudge() == 1)
    {
      JumpToIap= (IapFun)*(volatile uint32_t*)(FLASH_IAP_ADDR+4);
      __set_MSP(*(__IO uint32_t*) FLASH_IAP_ADDR);
      /*Jump to IAP program*/
      JumpToIap();
    }
  }
}
```

2.2.2 IAR Software Configuration

(1) Set the user code vector table offset address, the offset address should be consistent with the user code start address.

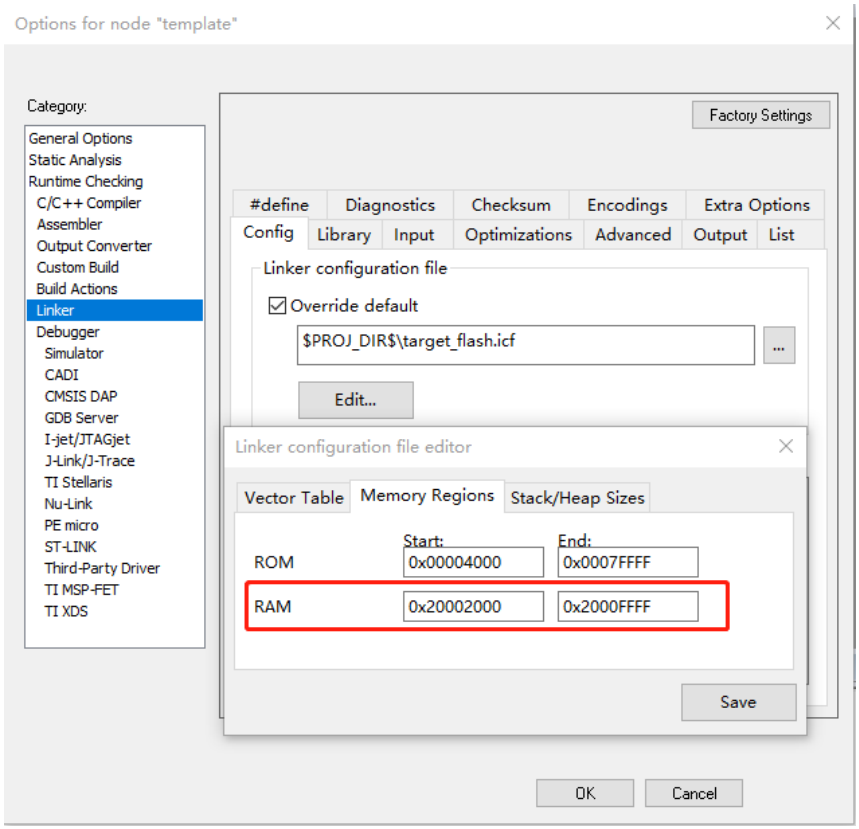


(2) If the user wants to use the debugger to observe the jump between IAP and APP, the ROM start address can be changed to the APP start address. (At this point, clicking the "Download and debug" button will download the IAP program directly to the area with a starting address of 0x4000.)

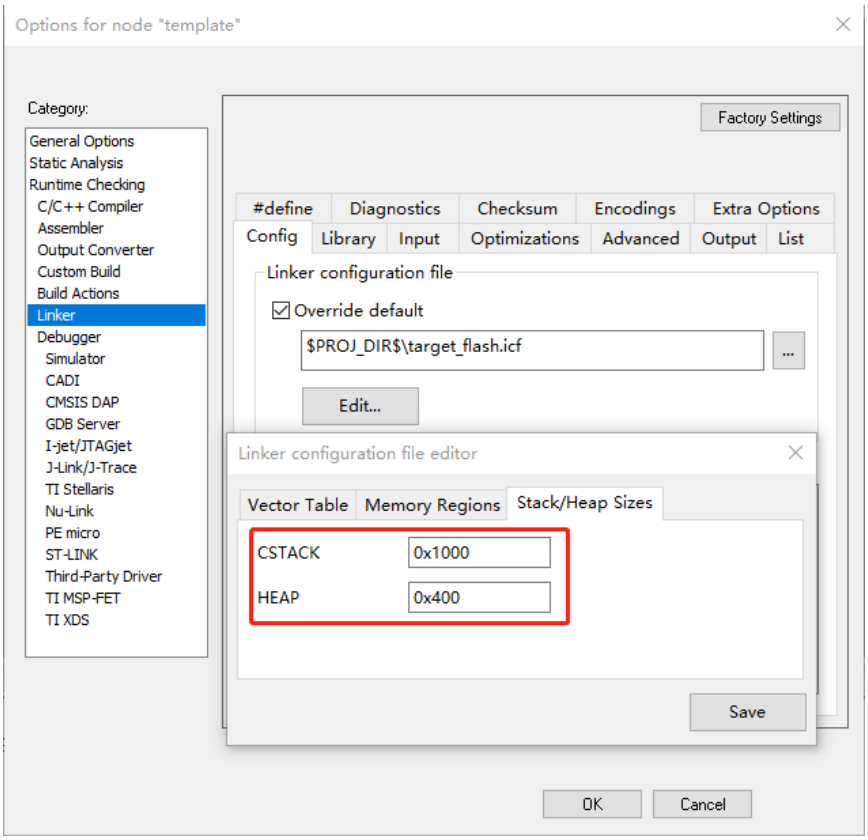


(3) Set the RAM size of the APP code.

Note: The RAM address space of the Bootloader and APP should be independent. Do not overlap. Otherwise, some variables may be overwritten.



(4) Set the stack size.



(5) During the execution of the user program, UART5 is checked. If UART5

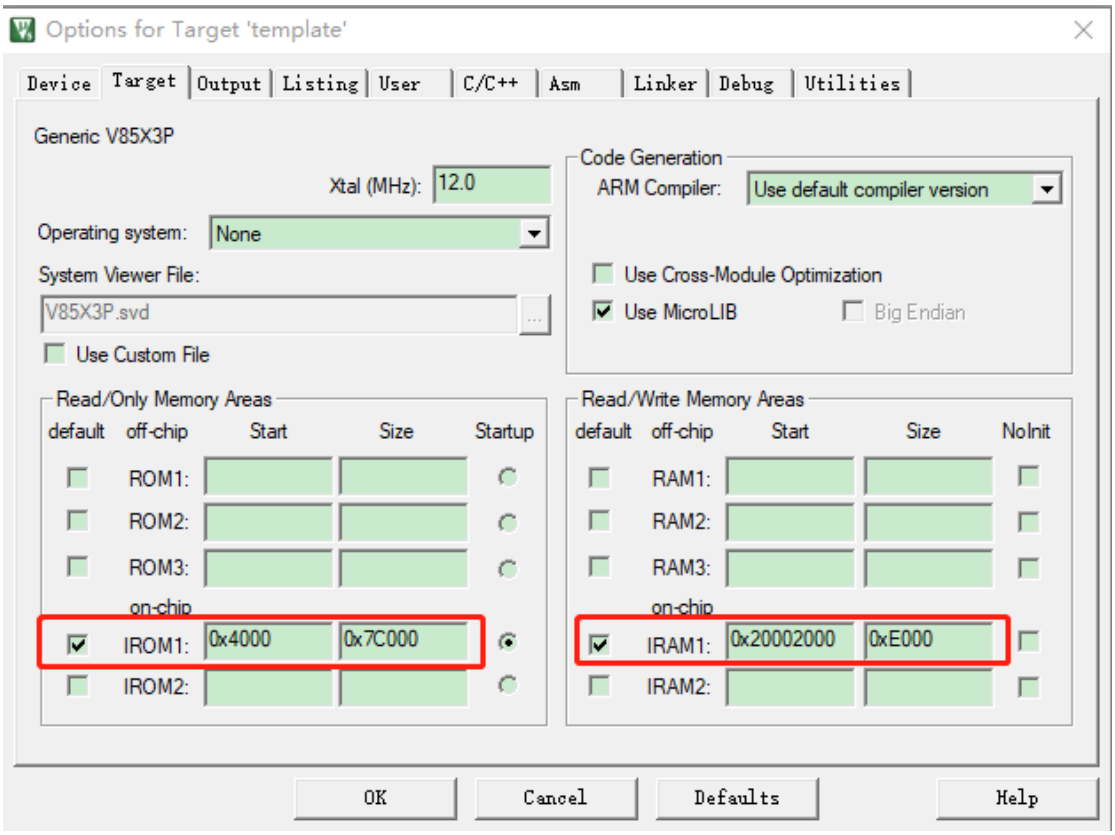
receives '6', it will jump to the IAP program for execution. (The conversion method can be controlled by itself)

2.2.3 KEIL Software Configuration

(1) Set the stack size (Note: the stack size needs to be set in the startup_target.s file).

```
Stack_Size      EQU      0x000001000
Heap_Size       EQU      0x00000400
```

(2) Set the user code ROM, RAM area size (click on the project, right click -> option)



3 Interrupt Vector Table

Figure 2 is a flow chart of the program with IAP.

（1） After target chip is reset, the address of the reset interrupt vector will be fetched from the address 0x00000004, and the corresponding reset interrupt service function will be executed.

（2） After the execution of the reset interrupt service function ends, jump to

the main function of the IAP;

（3） Execute the existing APP program or update the new APP to the flash area and execute the new APP program (set g_bAppRun to 1, g_bAppRun is defined in the IAP *main.c* file as a global variable), then after executing the IAP Jump to the interrupt vector table position of the APP (0x00000004+M), where M is the ROM offset position of the APP program;

（4） According to the position of the APP reset interrupt vector, jump to execute the reset interrupt service function of the APP, and then execute the main function of the APP;

（5） APP's main function is also a while (1) loop function. At this time, there are two interrupt vector tables in different positions in the flash area. However, when the main function is executed, if there is an interrupt request, the PC pointer is still forced to jump to the 0x00000000 interrupt vector table, instead of the interrupt vector table of the APP program.

（6） The program then jumps to the corresponding interrupt service function in the IAP according to the interrupt vector on the hardware. All 32 interrupt service functions in IAP are implemented as follows, ApplicationAddress is the application code start address, and ISR_HAND_OFFSET is the offset in the interrupt vector table (4 byte per unit, each interrupt vector is different). At this point, g_bAppRun=1, the program will jump to execute the interrupt service function of the APP;

```
typedef void (*pFun) (void);
void ISR_Handler(void)
{
  pFun p;

  if (g_bAppRun)
  {
      p = (pFun)(*(__IO uint32_t*)(ApplicationAddress + ISR_HAND_OFFSET));
      (*p)();
  }
  else
  {
      //BootloaderException during execution
  }
}
```

（7） After the execution of the APP interrupt service function ends, the program returns to the main function to continue execution.

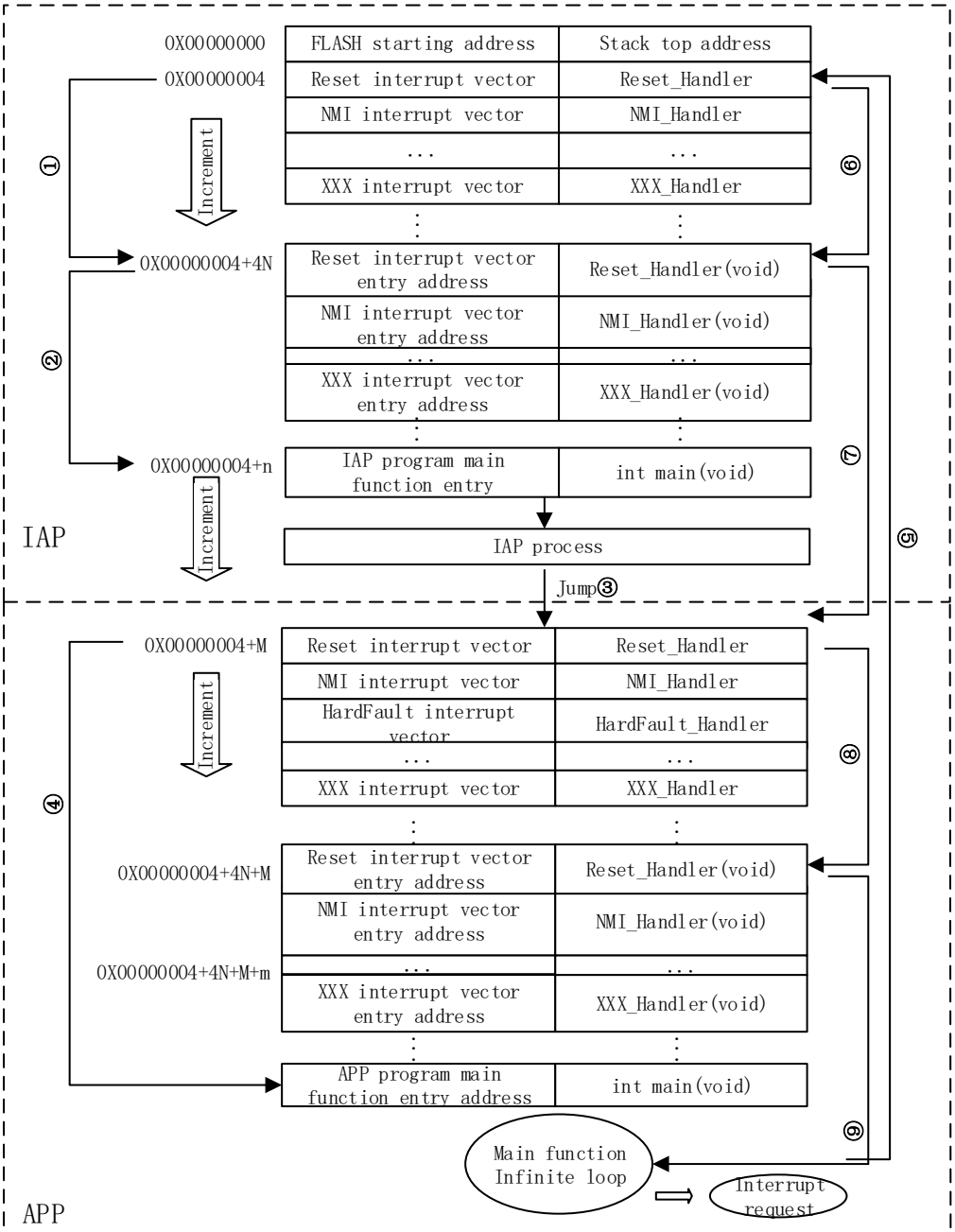


Figure 2. Program run flow chart

4 Run IAP Program

The IAP program is burned in the 0-511 sector of flash (1024bytes per sector). The user program occupies additional storage space.

4.1 Hyper Terminal Configuration

To use IAP, you need to run HyperTerminal on the PC. The configuration of HyperTerminal is as follows:

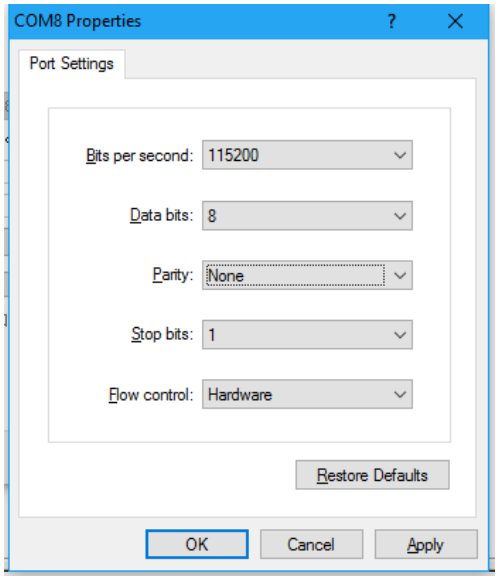


Figure 3. COM port properties

Note:

1. The baud rate used in the test is 115200.

2. When selecting the system clock frequency, in order to successfully use USART communication, the system clock on the chip must guarantee 115200 in baud rate.

4.2 Run IAP Program

Using the sample program in this article, press the button connected to the pin to start the IAP driver.

Pressing the button during reset allows the user to run the IAP driver to reprogram the target's internal flash memory. Of course, you don't have to use a button, you can also introduce an appropriate level on the pin. See Table 1 for details.

5 IAP Program Menu

Run IAP and display the following on the HyperTerminal:

The menu indicates that different functions will be performed when different numeric keys are pressed on the PC side.

- Press the 1 button on the PC to select the .bin file to download to the flash area.
- Press the 2 buttons on the PC to select the data in 0x4000-0x7FFFF in the

flash area and save it in the UploadedFlashImage.bin file. (The read area is determined by the macro defined in common.h - FLASH_IMAGE_SIZE)

- Press the 3 buttons on the PC to execute the user code.

5.1 Download image files to internal flash memory

Use HyperTerminal to download the binary file to the target internal flash memory. The steps are as follows:

1. Press 1 on the keyboard to select the “Download Image to the target Internal FLASH” menu.
2. Select Transfer->Send File in HyperTerminal.
3. In "Filename", enter the file name and path to be downloaded.
4. Select the "Ymodem" protocol in the protocol bar.
5. Click the "Send" button.

In this way, the IAP driver downloads the binary file from the defined base address into the internal flash memory of the target and displays the binary file name and size on the HyperTerminal window.

5.2 Read data from internal flash to .bin file

Use HyperTerminal to download the binary file to the target internal flash memory. The steps are as follows:

1. Press 2 on the keyboard to select the “Download Image to the target Internal FLASH” menu;
2. Select Transfer->Receive File in HyperTerminal;
3. Click “Browse” to select the folder and path where the received files are stored.
4. Select the "Ymodem" protocol in the protocol bar;
5. Click the "Receive" button;

In this way, the IAP driver will read the data in the internal flash memory of the target and save it to the corresponding binary file.

5.3 Execute a new program

Once the new program is downloaded, press the “Execute the New Program”

menu on the keyboard to execute the new program.

5.4 Unprotect write protection

When the IAP starts, it will detect whether the flash memory to be loaded by the user program is write-protected, and if so, it needs to be de-protected.

Write protection must be removed before downloading a new program. Write protection can be disabled by pressing 4 on the keyboard. A write reset will result in a system reset to reload the new option byte value.

6 Target chip IAP summary of implementation

The table below gives an overview of the IAP implementation.

Target chip		
IAP application	Address	0x00000000
	Number of pages of flash memory used	512 (page capacity: 1024 bytes)
User application		0x00004000(1)
Button (2)	Assigned pin	GPIOB.x
	Effective level	Low level
Flash program		For the target chip product, the programming program (program/erase) is executed from the flash memory.
USART peripherals used		UART5

Figure 4. target chip IAP implementation

Note:

（1） The starting address of the application is defined in the file common.h as follows:

```
#define ApplicationAddress 0x00004000
```

Modify this value to locate the user application to the desired address.

（2） Read the flash content from the inside and save it to the bin file. The read area is determined by the macro definition FLASH_IMAGE_SIZE in common.h:

```
#define FLASH_IMAGE_SIZE  (uint32_t) (FLASH_SIZE - (ApplicationAddress - 0x0000000))
```

Among them, FLASH_SIZE is the size of flash, and ApplicationAddress is the starting address of the user program.

（3） It is not necessary to use the button, you can introduce an appropriate level on the pin. This design sends '6' through the serial port 5.

7 User program rules

In a target chip microcontroller, the user must store the IAP application from the flash base address (0x00000000), while the sample user program starts at address 0x00004000.

IAP program is placed in the base address (0x00000000)of the flash, and APP program

8 How to use IAP driver

The IAP in the routine is a folder containing a series of subdirectories, as shown in Figure 5.

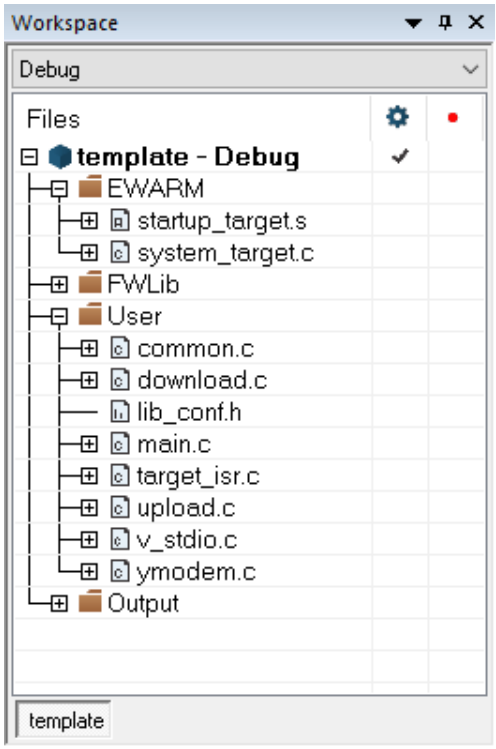


Figure 5. IAP driver directory structure

In order to use the IAP driver effectively

1. Use the development tool to download the IAP driver to the internal flash of

the target chip via the JTAG or SWD interface.

2. Open the HyperTerminal window and set the relevant parameters according to the settings defined in Section 4.1.
3. Press and hold the button on the target board to reset the IAP driver. The IAP menu will be displayed on the HyperTerminal window.
4. Press 1 and use the YMODEM protocol to download the application (as shown in Figure 5).

9 YMODEM Protocol

The YModem protocol can send 1024 bytes as a block of data: it is transferred to the data block of the target chip internal RAM and also performs error checking: comparing the transmitted and received data. A NAK response is to indicate that the data is not received successfully. If a NAK response occurs, the data block has not been successfully received.

The handshake process for transmitting data in the YModem protocol is as follows:

SENDER							RECEIVER
			——				C（commend：rb）
SOH	00	FF	foo.c	NUL[23]	CRC	CRC	——
			——				ACK C
SOH	01	FE	Data[128]		CRC	CRC	——
			——				ACK
SOH	01	FE	Data[128]		CRC	CRC	——
			——				ACK
SOH	01	FE	Data[100] CPEEOF[28]		CRC	CRC	——
			——				ACK
EOT							——
			——				ACK
SOH	00	FF	NUL[128]		CRC	CRC	——
			——				ACK
EOT							——
			——				ACK

Figure 6.YmodemProtocol handshake process

SENDER: sender.

RECEIVER: receiver.

The first step: the receiver sends a character 'C' to start the transmission, and

then waits for (SOH) state. If there is no response, it will time out and exit;

Step 2: After the sender receives the 'C', it sends the first data packet, as shown in the following table:

SOH 00 FF Foo.c NUL [123] CRC CRC

Table 2 Packet content parsing

byte	first	second	third	4th…131st (A total of 128 bytes)	132nd	133rd
Send data content	SOH	00	FF	Foo.c NUL[123]	CRCH	CRCL
	package header	package NO.	INV (NO.)	data	CRC check	
significance	Specify the size of the data area.	First package: 00 Second package: 01 The third package: 02 ..... Accumulated to: FF Then start incrementing from 00.	First package: FF Second package: FE The third package: FD ..... Decrease to: 00 Then cycle down from FF.	Data area size: 128 bytes.		

Note:

（1） The packet header is SOH, and the packet data area size is 128 bytes.
The packet header is STX, and the packet data area size is 1024 bytes.

（2） Only the data part participates in the CRC check, excluding the header and the code part.
The high byte of the checksum is the first, while the low byte is the second.

（3） If the file name is less than 128 bytes, fill it with 0.

Step 3: After receiving the first frame data packet, the receiver sends an ACK to indicate a successful data transfer, then sends 'C'.

Step 4: After receiving the ACK response and 'C', the sender starts to send the second packet. The data area in the second packet stores the 1024-byte data that needs to be sent.

Step 5: After receiving the data, the receiver sends an ACK and waits for the

next packet to be transmitted, followed by ACK/NAK.

Step 6: After the data transmission is completed, the sender sends an EOT, and the receiver responds with an ACK.

Step 7: The receiver will then send a 'C', and the sender will send a packet with all 0s, and in the beginning put "If there is no file to be transmitted. without the second file to be transmitted. SOH 00 FF 00~00 (128 total) CRCH CRCL, the receiver responds with an ACK.

Step 8: After the receiver answers the ACK, the data transmission is officially ended.

10 Related ASCII table

Table 3 ASCII table

Hex	Character	Hex	Character	Hex	Character
0	NUL	31	1	43	C
1	SOH	32	2	41	A
2	STX	33	3	61	a
6	ACK	34	4		
15	NAK	35	5		
18	CAN	36	6		